

What is a neural network

- Uma rede neural é um modelo matemático que aprende a mapear entradas (X) para saídas (Y) através de "neurônios" interconectados que conectam informações. Ao empilhar esses neurônios em camadas, a rede consegue identificar padrões complexos, reconhecer, como prever preços de imóveis com base em diversas variáveis.

Dados estruturados

- Dados organizados em tabela, com significado claro:
 - ↳ Cada coluna = uma informação específica
 - ↳ Tudo é bem definido e numérico (ou fácil de transformar)

Dados não estruturados

- Imagens, textos e áudios
- Não tem formato fixo, não são separados em colunas com significado claro e tem ambigüidade
 - ↳ pode ser um pouco de texto ou uma imagem

Binary Classification

Exemplos

classificar um gato = 1 - gato 0 - não gato

Pode ser imagem, porque uma matriz de uma coluna, com o conte de pixel x pixel x 3, que aí de o número de linhas.

X $\begin{bmatrix} x^1 & x^2 & x^3 \end{bmatrix}$ cada coluna = uma imagem

1	1	1
1	1	1
1	1	1

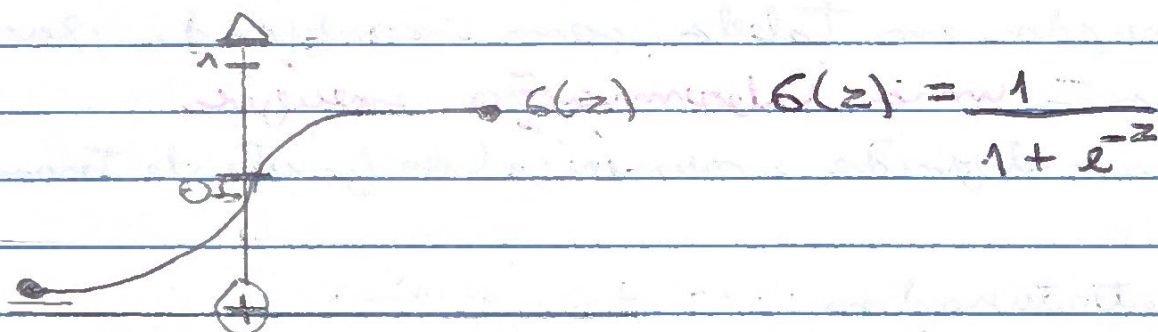
D	S	T	Q	Q	S	S
D	L	M	M	J	V	S

$$Y = [y^1 y^2 \dots y^n]$$

Sigmoid Regression

Given x , $\hat{y} = P(Y=1|x)$
 $x \in \mathbb{R}^n$ $0 \leq \hat{y} \leq 1$

Output $\hat{y} = \sigma(\underbrace{w^T x + b}_z)$



If z large, $\sigma(z) \approx \frac{1}{1+0} = 1$

If z large negative number, $\sigma(z) \approx \frac{1}{1+e^{-z}} \approx \frac{1}{1+\text{large}} \approx 0$

Levada Gradiente

1) pass imagem (x)

2) Calcular

$$z = w \cdot x + b$$

3) Calcular erro:

$$L(\hat{y}, y)$$

4) Juntar tudo:

$$J(w, b)$$

5) Atualizar:

lendaia

$$w := w - \alpha \frac{dJ}{dw}$$

$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

$$\mathcal{L}(a, y) = -(y \log(a) + (1-y) \log(1-a))$$

$$\begin{array}{l}
 x_1 \\
 w_1 \\
 x_2 \\
 w_2 \\
 b
 \end{array}
 \rightarrow
 z = w_1 x_1 + w_2 x_2 + b \rightarrow a = \sigma(z) = \mathcal{L}(a, y)$$

$$\frac{da}{dz} = a(1-a) \quad "da" = \frac{d\mathcal{L}(a, y)}{da}$$

$$dz = \frac{dw_1}{dz} \cdot da + \frac{dw_2}{dz} \cdot da = \frac{1}{a} + \frac{1}{1-a}$$

$$dz = a - y$$

$$dw_1 = x_1 \cdot dz$$

$$w_1 = w_1 - \alpha \cdot dw_1$$

$$dw_2 = x_2 \cdot dz$$

$$w_2 = w_2 - \alpha \cdot dw_2$$

$$db = dz$$

$$b = b - \alpha \cdot db$$

Gradient descent on m examples

$$J = 0; \quad dw_1 = 0; \quad dw_2 = 0; \quad db = 0;$$

$$\text{for } (i = 1; i \leq m; i++) \{$$

$$z = w_1 * x_1[i] + w_2 * x_2[i] + b;$$

$$a = \text{sigmoid}(z);$$

$$J += -(y[i] * \log(a) + (1 - y[i]) * \log(1 - a));$$

$$dw_1 += x_1[i] * dz; \quad dw_2 += x_2[i] * dz;$$

$$db += dz;$$

$$J /= m; \quad dw_1 /= m; \quad dw_2 /= m; \quad db /= m;$$

$$h = h - \alpha \nabla h_i$$

Veterinária

- Poder de pensar em elementos por elementos e pensar em um "sector inteiro de uma vez"

$$Z = [z^{(1)} \ z^{(2)} \ \dots \ z^{(i)}] = w^T X + [b \ b \ \dots \ b] = w^T X + b \quad w^T x + b \dots$$

$$= z = \text{np.dot}(w.T, x) + b$$

Broadcasting

- O número "ético" virtualmente é maior menor ou seja ele coincide com a forma da maior matriz

* Examples

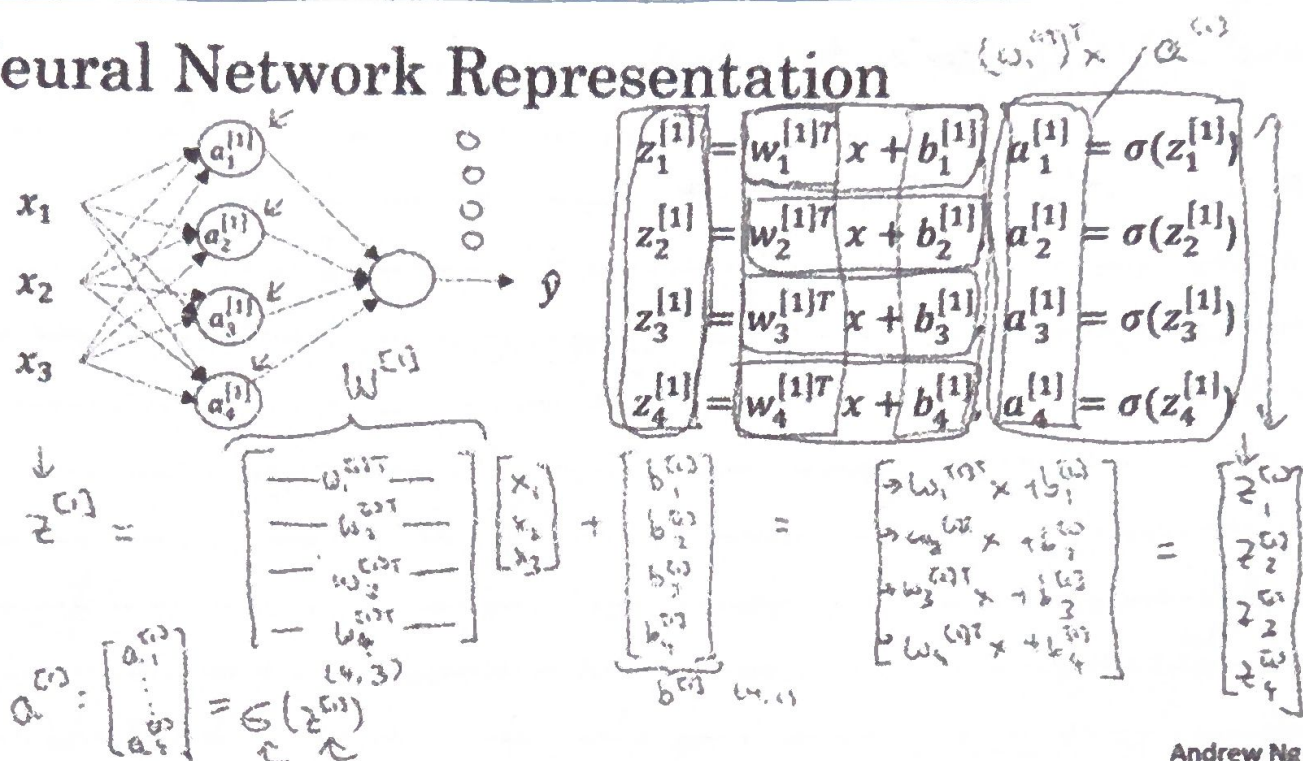
- matriz (m, n) + escalar $(1, 1)$ = o escalar é somado a todos elementos da matriz

- matriz (m, n) + vetor linha $(1, n)$: o vetor linha é aplicado em todas as linhas da matriz

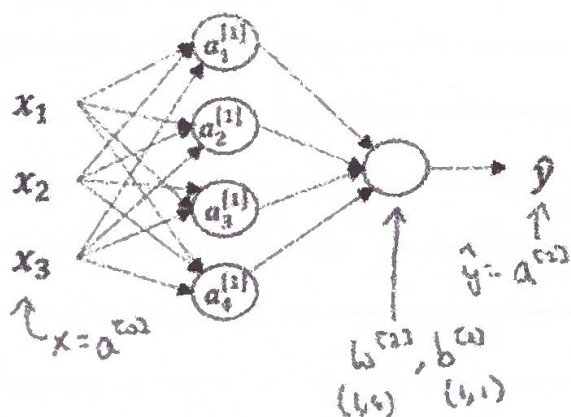
• $\text{matriz}(m, n) + \text{vetor coluna}(n, 1) = \text{se vetor coluna é replicado } n \text{ vezes para as linhas}$

Comando	shape	tipo	Recomendacoes
<code>np.random.randn(S)</code>	$(S,)$	Rank 1 array	none
" " "	$(S,1)$	vector column	sum
" " "	$(1,S)$	vector linha	sum

Neural Network Representation



Neural Network Representation learning



Given input x :

$$\rightarrow z^{[1]} = W^{[1]}x + b^{[1]}$$

(4,1) (4,3) (3,1) (4,1)

$$\rightarrow a^{[1]} = \sigma(z^{[1]})$$

(4,1) (4,1)

$$\rightarrow z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

(1,1) (1,4) (4,1) (1,1)

$$\rightarrow a^{[2]} = \sigma(z^{[2]})$$

Sigmoid: varia entre 0 e 1. números grandes e pequenos são muito lentos.

Tanh: varia entre -1 e 1. É quase sempre melhor do que Sigmoid para as camadas ocultas porque a média das saídas é próxima a 0, o que é mais rápido.

Relu: para valores positivos de z , a inclinação é sempre 1.

Leaky Relu: dá uma pequena inclinação aos valores negativos (0.01), garantindo que a neurona nunca "morre" completamente.

Why non-Linear Activation Functions

Problema de ativação linear

Se você usar uma função de ativação linear, a rede neural apenas computará uma função linear dos dados de entrada.

Composição de funções

A composição de funções lineares é apenas uma função linear. Logo, não importa quantas camadas ocultas tenha, ele não será mais expressivo do que um modelo linear simples.

Regra Geral

mesmo com problemas de regressão, as camadas ocultas devem sempre utilizar funções não lineares como ReLU, Tanh ou Sigmoid.

Derivates of Activation Function

Função Sigmoid

Fórmula: $g(z) = \frac{1}{1 + e^{-z}}$

Derivada: É calculada como $g(z)(1 - g(z))$

Simplificação: $a = g(z)$ $a(1 - a)$

Comportamento: Quando z é muito grande ou muito pequeno, a inclinação é próxima de 0. O valor máximo da derivada ocorre em $z=0$, sendo igual a 0,25.

Função Tanh

- Fórmula: $g(z) = \tanh(z)$
- Derivada: $g'(z) = 1 - (g(z))^2$
- Simplificação: Se $a = g(z)$, então $g'(z) = 1 - a^2$
- Comportamento: assim como na rede de inclinação, vai a 0 quando z é muito grande ou pequeno. Em $z=0$, a inclinação é 1

Função ReLU

- Fórmula: $g(z) = \max(0, z)$
- Derivada: $g'(z) = 0$ se $z < 0$
1, se $z > 0$

Obs: em $z=0$, a derivada não é tecnicamente definida, mas na prática de programação costuma-se atribuir 0 ou 1 sem problemas aos componentes do algoritmo.

Função Leaky ReLU

- $g(z) = \max(0, 0.1z, z)$
- Derivada: $g'(z)$
 - 0,01 se $z < 0$
 - 1 se $z > 0$

Back Propagation

Objetivo

Queremos ajustar os pesos (W) e bias da rede diminuindo o erro da rede

2. Forward Pass

$$z^1 = W^{(0)}x + b^1$$

$$z^2 = W^1 \cdot a^1 + b^2$$

$$a^1 = g(z^1)$$

$$a^2 = g(z^2)$$

3. Erro

$$\text{erro} = a^2 - y$$

$$dz^2 = \text{erro da neurônio de saída}$$

$$dz^2 = a^2 - y$$

4. Gradientes da Última Camada

$$dW^2 = (1/m) \cdot dz^2 \cdot a^{1T}$$

$$db^2 = (1/m) \cdot \text{erro} \cdot dz^2$$

$$dW^2 = \text{erro} \times \text{entrada}$$

$$db^2 = \text{média dos erros}$$

5. Voltando o erro (Back Prop)

$$dz^1 = (W^{2T} \cdot dz^2) * g'(z^1)$$

$$W^{2T} \cdot dz^2 = \text{espalha o erro para a camada anterior}$$

$$g'(z^1) = \text{ajuste pela função de ativação}$$

$$dz^1 = \text{erro da camada recuada}$$

6. Gradientes da Função Custo

$$dW^1 = (1/m) \cdot dZ^1 \cdot X^T$$

$$db^1 = (1/m) \cdot \text{suma}(dZ^1)$$

$$dW^1 = \text{erro} \times \text{entrada}$$

$$db^1 = \text{média dos erros}$$

9. Atualização dos Parâmetros

$$W = W - a \cdot dW$$

$a = \text{learning rate}$

$$b = b - a \cdot db$$

8. Significados das variáveis

$$dZ = \text{erro dos neurônios}$$

$$dW = \text{quanto ajustar o peso}$$

$$db = \text{quanto ajustar o bias}$$

11 - - - - -

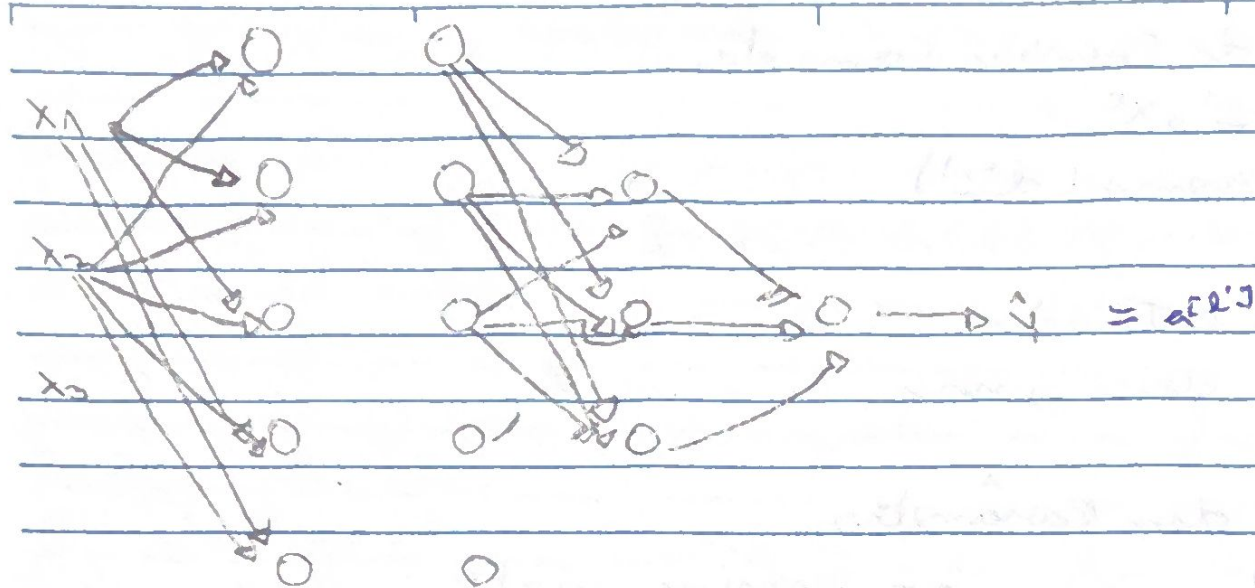
Perceptron Simples

- pode iniciar com zero
- só tem um neurônio

Rede neural

- NÃO pode iniciar com zero
- antes todos neurônios ficam iguais

D	S	T	Q	Q	S	S
D	L	M	M	J	V	S



$$n^{[1]} = 3 \quad n^{[2]} = 5 \quad n^{[3]} = 1$$

$L = 3 \rightarrow$ número total de camadas na rede

$n^{[l]} \rightarrow$ unidades na camada l

$a^{[l]} \rightarrow$ ativações na camada l

matriz dimensões

$Z =$ número de neurônios - por 1 - ex $[3, 1]$

$$W^{[l]} = (n^{[l]}, n^{[l-1]})$$

$$b^{[l]} = (n^{[l]}, 1)$$

$$dW^{[l]} = (n^{[l]}, n^{[l-1]})$$

$$db^{[l]} = (n^{[l]}, 1)$$

|| - - - - -

$$DZ^{[l]}, A^{[l]} = (n^{[l]}, m)$$

$$l=0 \quad A^{[0]} = X = (n^{[0]}, m)$$

$$DZ^{[l]}, DA^{[l]} = (n^{[l]}, m)$$

envolve o número de exemplos

Parâmetros vs. Hiperparâmetros

Parâmetros

- São as variáveis que o algoritmo de aprendizado ajusta automaticamente durante o treinamento, especificamente W e b

Hiperparâmetros

- São as configurações que serão passadas ao algoritmo de aprendizado e que determinam como os parâmetros (W e b) serão calculados, como:

- taxa de aprendizado
- número de iterações do gradiente descendente
- número de camadas ocultas
- número de unidades ocultas em cada camada

forward and backward propagation

$$\begin{aligned} Z^{[1]} &= W^{[1]}X + b^{[1]} \\ A^{[1]} &= g^{[1]}(Z^{[1]}) \\ Z^{[2]} &= W^{[2]}A^{[1]} + b^{[2]} \\ A^{[2]} &= g^{[2]}(Z^{[2]}) \\ &\vdots \\ A^{[L]} &= g^{[L]}(Z^{[L]}) = \hat{Y} \end{aligned}$$

$$\begin{aligned} dZ^{[L]} &= A^{[L]} - Y \\ dW^{[L]} &= \frac{1}{m} dZ^{[L]} A^{[L]T} \\ db^{[L]} &= \frac{1}{m} \text{np.sum}(dZ^{[L]}, \text{axis} = 1, \text{keepdims} = \text{True}) \\ dZ^{[L-1]} &= dW^{[L]T} dZ^{[L]} g'^{[L]}(Z^{[L-1]}) \\ &\vdots \\ dZ^{[1]} &= dW^{[L]T} dZ^{[2]} g'^{[1]}(Z^{[1]}) \\ dW^{[1]} &= \frac{1}{m} dZ^{[1]} A^{[1]T} \\ db^{[1]} &= \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True}) \end{aligned}$$

Train / Dev / Test

machine Learning é iterativa

• Seleção → código → teste → ajuste → repetir

Training set

• Usado para o modelo aprender os padrões

Dev set

• Usado para testar modelos e ajustar hiperparâmetros

Test set

• Usado no final para avaliar o desempenho real

// - - - - -

Divisão dos dados

* Anteriormente

• 70/30 ou 60/20/20

* Hoje (Big Data)

• 99% Treino / 1% dev / 1% Teste

Diagnósticos através de métricas

Erro de Treino	Erro de Teste	Diagnóstico	Porquê?
1%	11%	Alta Variância	O modelo decorou o treino, mas não generaliza
15%	18%	Alta viés	O modelo nem sequer aprendeu bem os dados de treino
15%	30%	Viés e variância altos	Não aprendeu bem e ainda falha na generalização
0.5%	1%	Baixo viés e variância	O cenário ideal

OBS: uma observação importante é que a "alta viés" é relativa. Se as imagens têm ruído que nem um humano consegue distinguir (ex: 15% de erro humano), então um erro de treino de 15% não é considerado "alt viés", mas sim o limite ideal do que é possível alcançar.

Pequena Prática para Aprendizagem de máquina

Passo 1: O modelo tem Alta viés?

- tente uma rede neural maior
- treine o modelo por mais tempo
- experimente algoritmos de otimização avançados
- tente uma arquitetura de rede diferente

D	S	T	Q	Q	S	S
D	L	M	M	J	V	S

Como a: O modelo tem alta variância

- Obter mais dados
- Regularização: usa técnicas para reduzir o overfitting

Regularização L2

Solução

• Punição pesos grandes

Os modelos com pesos grandes = memoriza demais = overfitting

Fórmula

$$J = J_{\text{original}} + \frac{\lambda}{2m} \sum v^2$$

J_{original} = erro normal do modelo

$\sum v^2$ = para todos os pesos da rede, elevar ao quadrado e soma tudo

λ = controla a força de punição

alta = pesos menores $\rightarrow$ modelo simples

baixa = pesos livres $\rightarrow$ modelo complexo

$$w = w - \alpha (dw + \frac{\lambda}{m} w)$$

Why Regularization Reduces Overfitting

1) Simplificação de Rede Neural

A regularização adiciona um termo de penalidade à função de custo que desestimula pesos (W muito grandes).

Intuição: Ao aumentar o parâmetro de regularização (λ), os pesos são forçados a ficar próximos a 0.

Efeito: isso reduz o impacto de várias unidades ocultas. Embarca não se "desligue" completamente, torna a rede mais simples, comportando-se quase como uma rede menor ou uma regressão logística, o que reduz a variância e o overfitting.

2) Ativação Linear e Redução da Complexidade

↳ **Regime Linear:** se o parâmetro λ for grande, os pesos W serão pequenos. Como $z = W \cdot A + b$, se W for pequena, z também será pequena.

↳ **Linearidade:** para valores pequenos de z , a função tanh opera em uma região quase linear.

↳ **Resultado:** se todas as camadas operarem de forma aproximadamente linear, a rede inteira se comportará como um modelo linear simples. Modelos lineares não conseguem apertar fronteiras de decisão hiper-complexas, o que impede que a rede decore o ruído dos dados de treinamento.

Monitoramento: é para verificar se o algoritmo está convergindo corretamente, você deve plotar a nova função de custo J com penalidade. Se você plotar apenas a função de custo original, poderá não ver uma queda constante e monotônica a cada iteração.

DropOut Regularization

1) Criando a máscara (quem fica ligado ou desligado)

$d3 = \text{np.random.rand}(\text{a3.shape}[0], \text{a3.shape}[1]) < \text{keep_prob}$

• **Seu caso uma matriz $d3$ com valores:**

$\text{True}(1) \rightarrow$ neurônio **fica**

$\text{False}(0) \rightarrow$ neurônio **é desligado**

2) Aplicando a máscara

$a3 = \text{np.multiply}(a3, d3)$

Onde $d3 = 0$, **seu 0 o neurônio**

Onde $d3 = 1$, **continua igual**

• **"mata" 20% dos neurônios**

3) Inverted Dropout

$a3 /= \text{keep_prob}$

Como desliga os neurônios, o valor médio de ativação **cain**

sem Dropout

• sempre a mesma rede $\rightarrow$ tende a **deconar**

Com dropout

• Rede muda toda hora $\rightarrow$ não consegue **deconar fácil**

Seu **gerar** a rede a não depende de neurônios **específi**

cos **especificando** **qualquer** **modo** **gerar**

de **gerar** **uma** **rede** **com** **uma** **estrutura** **específi**

ca **uma** **maneira** **específi** **uma** **rede** **de** **atividades** **uma** **rede**

de **atividades** **uma** **rede** **de** **atividades** **uma** **rede**

de **atividades** **uma** **rede** **de** **atividades** **uma** **rede**

Dropout por camada

* Camadas grandes

- Usar keep-prob ≈ 0.5
- mais regularização (maior risco de overfitting)

* Camada menores

- Usar keep-prob $\approx 0.7 - 1.0$
- menos regularização

* Camada de entrada

- Usar ≈ 1.0 (ou 0.9)
- não perder informação importante

Normalização de entradas

O que é?

- A normalização é uma técnica de pré-processamento de dados que coloca todos as características no mesmo escala

subtração e média:

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)} \quad x_0 = x - \mu$$

normalizar e variância

$$\sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)})^2 \quad x_0 = \frac{x}{\sigma}$$

dados normalizados criam um caminho curto e reto para o aprendizado. dados brutos criam um caminho longo, torto e cheio de obstáculos.

D	S	T	Q	Q	S	S
D	L	M	M	J	V	S

$$W^{[L]} = \text{np.random.randn}(\text{dshape}) * \text{np.sqrt}(\frac{2}{n^{(L-1)}})$$

• Criação que o valor de 2 explode ou não em redes profundas

• Truque para compensar o ReLU + que ignora os números negativos.

Numerical Approximations of Gradients

$$\frac{f(0+\epsilon) - f(0-\epsilon)}{2\epsilon}$$

Adicionalmente pode ser aproximado usando a diferença central, que calcula a inclinação de funções em um ponto θ . Nesse caso escolhemos um valor pequeno ϵ . Uma aproximação que muito próxima a derivada real

Gradient checking

Preparar $W^{[L]}, b^{[L]}, \dots, W^{[1]}, b^{[1]}$ and reshape into a big vector θ
Preparar $dw^{[L]}, db^{[L]}, \dots, dw^{[1]}, db^{[1]}$ and reshape into a big vector $d\theta$

$$d\theta_{\text{approx}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon}$$

$$\approx d\theta[i] = \frac{\partial J}{\partial \theta} \quad \left| \quad d\theta_{\text{approx}} \approx d\theta \right.$$

check

$$\|d\theta_{\text{approx}} - d\theta\|_2 \approx 10^{-9} \text{ great}$$

$$\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2 \approx 10^{-5}$$

$$10^{-3} \text{ bad}$$

$$\epsilon = 10^{-9}$$

mini Batch

na mini-batch, você coloca várias imagens na rede ao mesmo tempo.

A rede gera uma saída para cada imagem, não uma só.

ex: 20 imagens, 20 respostas

Depois, compara cada resposta com o valor real, calcula o erro de cada uma, tira a média dos erros

Com essa média você atualiza os pesos uma única vez.

Gradient Descent With momentum

$$v_{den} = \beta v_{den} + (1 - \beta) \Delta_{den}$$

$$v_{db} = \beta v_{db} + (1 - \beta) \Delta_{db}$$

$$w = w - \alpha v_{den}, \quad b = b - \alpha v_{db}$$

Rms-prop

$$S_{den} = \beta_2 S_{den} + (1 - \beta_2) \Delta_{den}^2$$

$$S_{db} = \beta_2 S_{db} + (1 - \beta_2) \Delta_{db}^2$$

$$w_0 = w - \alpha \frac{\Delta_{den}}{\sqrt{S_{den} + \epsilon}}, \quad b_0 = b - \alpha \frac{\Delta_{db}}{\sqrt{S_{db} + \epsilon}}, \quad \epsilon = 10^{-8}$$

Adam optimisation algorithm

$$V_{den} = 0 \quad S_{den} = 0 \quad V_{db} = 0 \quad S_{db} = 0$$

$$V_{den} = \beta_1 \cdot V_{den} + (1 - \beta_1) \cdot den, \quad V_{db} = \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$$

$$S_{den} = \beta_2 \cdot S_{den} + (1 - \beta_2) \cdot den^2, \quad S_{db} = \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

$$V_{den}^{corrected} = \frac{V_{den}}{(1 - \beta_1^*)}$$

$$V_{db}^{corrected} = \frac{V_{db}}{(1 - \beta_1^*)}$$

$$S_{den}^{corrected} = \frac{S_{den}}{1 - \beta_2^*}$$

$$S_{db}^{corrected} = \frac{S_{db}}{1 - \beta_2^*}$$

$$V := V - d \cdot \frac{V_{den}^{corrected}}{\sqrt{S_{den}^{corrected}} + \epsilon}$$

$$b := b - \frac{V_{db}^{corrected}}{\sqrt{S_{db}^{corrected}} + \epsilon}$$

d : needs to be Time

$$\beta_1 = 0.9 \quad (den)$$

$$\beta_2 = 0.999 \quad (den^2)$$

$$\epsilon = 10^{-8}$$

// - - - - -

(o que é o V (momentum))

o termo V é uma média móvel dos gradientes

↳ **Acumula a direção das atualizações ao longo do tempo.**

Funções:

• suavizar oscilações

• Reduzir ruído do gradiente

• manter uma direção consistente de descida

V representa a direção média de aprendizagem, sem ele

o modelo pode ficar superaquecendo e demorar mais para convergir.

o que é o S (RMS map)

- o termo S é uma média móvel dos gradientes ao quadrado.
- ele mede a magnitude (tamanho) dos gradientes.
- o termo S é ajustado automaticamente.
- evitar pesos muito grandes ou muito pequenos.

o que dividir por $\sqrt{2}$?

gradiente grande $\rightarrow S$ grande $\rightarrow$ peso diminui
gradiente pequeno $\rightarrow S$ pequeno $\rightarrow$ peso aumenta

V = boa direção e pesos grandes

deu / de digam quanto mudar

- $V \rightarrow$ moving ins
- $S \rightarrow$ regular ins

Hiperparâmetros, tuning e escala

Escala Linear

quando usar: quando o impacto das mudanças no parâmetro é proporcional ao tamanho de toda a interação

Exemplos:

- número de unidades ocultas ($n^{[2]}$): ex de 50 a 100
- número de camadas (L): ex: de 2 a 5

colher 60 ou 70 tem um impacto linear e matemático similar no modelo

Grada Socioeconômica

↳ Quando usar: quando o parâmetro varia em ordem de magnitude (potências de 10)

* Taxa de aprendizado

- Se você buscar α entre 0,0001 e 1 de forma linear, 90% das suas tentativas estarão entre 0,1 e 1,0
- É ineficiente, pois valores menores (0,0001 e 0,01) costumam ser muito importantes para convergência

Para $\alpha \approx 0$, quando ele varia de 0,9 a 0,999, pequenas mudanças, tem impactos gigantes

// - - - - -

Batch norm

Dados uma camada com valores intermediários $z^{(1)}, \dots, z^{(m)}$

1) Calcular a média

$$\mu = \frac{1}{m} \sum z^{(i)}$$

2) Calcular a variância

$$\sigma^2 = \frac{1}{m} \sum (z^{(i)} - \mu)^2$$

3) Normalizar

$$z_{\text{norm}} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

4) Escalar e deslocar

$$\hat{z}^{(i)} = \gamma z_{\text{norm}} + \beta$$

↳ parâmetros aprendidos para normalizar e reverter
para valores de 0 e 1

Problema Recorrente

$$x = e^{(z^{EL})}$$

$$a_i^{EL} = \frac{e^{z_i^{EL}}}{\sum x_i}$$

$$a_i^{EL} = \frac{x_i}{\sum x_i}$$

$z^{EL} =$	5
	2
	-1
	3

$x = e^z =$	e^5	$\approx 148,4$
	e^2	$\approx 7,4$
	e^{-1}	$\approx 0,37$
	e^3	$\approx 20,1$

$$\sum x_i = 176,3$$

$$a_i = \frac{x_i}{176,3}$$

$$a_1^{EL} = \frac{148,4}{176,3} = 0,842$$

$$a_3^{EL} = \frac{0,37}{176,3} = 0,002$$

$$a_2^{EL} = \frac{7,4}{176,3} = 0,042$$

$$a_4^{EL} = \frac{20,1}{176,3} = 0,114$$

$a^{EL} =$	0,842
	0,042
	0,002
	0,114

Train / Dev / Test set distribution

Alinhamento de distribuição

- O erro mais comum é criar conjuntos de desenvolvimento e de teste que possuem distribuições diferentes

Recomendação: A análise ideal é misturar todos os dados disponíveis e distribuí-los aleatoriamente entre os conjuntos de desenvolvimento e de teste, garantindo que ambos reflitam a mesma distribuição de dados que você espera encontrar no futuro.

Human Level Performance

Bayes Optimal Error (erro bayesiano ótimo)

é o limite teórico máximo de precisão para uma tarefa impossível superar esse erro, pois ele representa o "melhor desempenho possível"

→ a maioria tende a se aproximar até atingir o nível humano, mas costuma decrescer após atingi-lo

Iteração do nível humano

→ menor margem de erro

→ o desempenho humano se está próximo ao limite teórico, salvando áreas para melhorias verificáveis.

→ Simulação de Ferramentas

→ Ferramentas de diagnóstico, como rotulagem por humanos, análise de erros e a apuração das causas e variáveis, tornam-se menos eficazes em casos complexos de aplicação quando a máquina se é superior

Reducing (avoidable) bias and variance

Human-level



Avoidable bias

Linear logistic model

Linear logistic / better optimization algorithms

- momentum, RMS Prop, Adam

NN architecture / hyperparameter search

Training error



variance

more data

Regularization (L1, dropout)

NN architecture / bias

Análise de Erros

- Criamos manual de algar e exemplos que o modelo erra no conjunto de dev
- Tome os erros e classifique-os em categorias. Se apenas 5% dos erros são de "leitura", o máximo que você pode fazer o modelo ganhando mais é 0,5% (em um erro total de 10%) - foque na categoria que cause mais erros.

Dados com Rotulos Incorretos

- O Deep Learning é robusto a erros aleatórios no treino, mas não faz um erro sistemático, pois pode deixar como está o volume de dados por grande.
- ↳ no dev / teste é mais crítico?
- Se a porcentagem de erros por rótulo errada for grande em relação ao erro total, corrija-os.
- Se corrigir o dev, corrija o teste também para manter a mesma distribuição.

Distribuições Diferentes

- O problema: você tem muitos dados de web (limpos / perfeitos) e poucos dados de usuário (ruins / reais)
- O conjunto de dev e teste deve ser de mesma distribuição e representar o que você quer no mundo real
- * Como dividir os dados:
 - ↳ Treino: pode misturar (dados de web + alguns dados reais)
 - ↳ Dev e Teste: use apenas os dados reais de usuário
- O dev set é o seu alvo. Se você usar botões de web no dev, seu modelo ficará ótimo em botões perfeitos, mas falhará no celular do usuário

D S T O O S S
 D L M M I V S

Análise de Erros: Bias, Variância e Overfitting

Quando os dados de treino e teste vêm de fontes diferentes (ex: fontes de internet vs. fontes de celular), usamos um **Training Set** para não confundir erros de generalização com erros de dados diferentes.

Erros Aleatórios ↑
Erros de Treino ↓
 Evitável → o modelo não aprendeu nem o **Bias** único → modelo nunca / não cometeu nem mais tempo

Variância = Erro do Training Set - Erro de Treino

• O modelo decorou o treino (**overfitting**) e não generaliza para dados novos da mesma fonte

• Ação: Regularização (L2, Dropout) ou mais dados de treino

Data Mismatch (dados incompatíveis) = Erro de teste - Erro do Training Set

• O modelo aprendeu a tarefa, mas não funciona na **distribuição real do mundo** (de teste)

• Ação: coletar treinos mais variados com o teste ou usar **intérm de dados**

Overfitting ao teste = Erro de teste - Erro de teste

• Você mexeu tanto nos parâmetros para o teste ficar bom que o modelo "fixou nele"

• Ação: aumentar o tamanho do teste

Training set = treino

Test set = teste mundo real

Training data = teste generalização Test set = validação geral

(mesmo tipo de dados)

Incremental Seemings

O que é:

- Usar o conhecimento de uma Target A (ex: reconhecer objetos comuns) para ajudar uma Target B (ex: diagnósticos com Raios-X)
- ↳ como fazer:

- **Excluir a última camada da rede original**
- **Adicionar uma nova camada de rede com pesos aleatórios**
- **Fazer a fine-tuning:** re-treinar a rede com novos dados
 - ↳ Quando novo: quando você tem muitos dados para a Target A, mas poucos dados para a Target B, e as entradas (X) são do mesmo tipo (ambas são imagens, por exemplo)

Multitask Seemings

O que é:

- Treinar uma única rede para realizar várias tarefas simultaneamente

Diferença do Softmax

- no multitask, uma única imagem pode ter múltiplos rótulos positivos (ex: pode ter um carro e uma placa de pare no mesmo foto)

Vantagem:

- As tarefas compartilham características de baixo nível (bordes, formas), o que melhora a performance geral

Desafios incompletos

- Funcionar mesmo se algumas não tiverem todos os rótulos ou exemplos

End-to-End Deep Learning

o que é:

- substituir um sistema de várias etapas (pipeline) por uma única rede neural que mapeia a entrada direta para a saída final.

exemplos:

- Áudio → transcrição (sem precisar passar por fonemas ou extração manual de áudio)

Crus:

- deixar os dados "falarem" livremente e simplificar o design do sistema

Contras:

- Exige uma quantidade massiva de dados para funcionar bem

Quando não usar:

- Quando você tem poucos dados ou quando o problema pode ser quebrado em subproblemas mais simples (como no reconhecimento facial: primeiro detecta o rosto, depois identifica e pensa)

Padding

o que é?

- Adicionar bordas artificiais (geralmente zeros) ao redor da imagem antes da convolução

1 2 3

0 0 0 0 0

evitar que a imagem diminua rápido

4 5 6

0 1 2 3 0

Preservar informações das bordas

7 8 9

0 4 5 6 0

0 7 8 9 0

0 0 0 0 0

$$p = \frac{b-1}{2}$$

o calcular o valor de padding p e o valor do kernel

Stride

o que é?

- O tamanho da janela que o kernel desloca ao andar pela imagem

efeito principal

- Quanto maior o stride, menor fica o número de saída.

$$\frac{n + 2p - k}{s} + 1$$

n = tamanho da entrada
 p = padding

k = tamanho do filtro
 s = stride

D	S	T	D	Q	S	S
P	L	M	M	J	V	S

Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$m^{[l]}$ = number of filters

Input: $n_H^{[l-1]} \times m_H^{[l-1]} \times m_C^{[l-1]}$

Output: $n_H^{[l]} \times m_H^{[l]} \times m_C^{[l]}$

$$\frac{1-2}{5} = -0.4$$

$$1 + \frac{1-2}{5} = 0.6$$